Phase 1 Report — Student Completion Verification and Digital Letter Issuance System

CSI473 Software Engineering · Semester 1, 2026/27 · University of Botswana

Draft — Laboratory 5

1. Introduction

This report consolidates the Phase 1 analysis for the Student Completion Verification and Digital Letter Issuance System. It draws together the approved problem, the requirements, the use cases, the domain analysis and the behavioural models produced across Laboratories 1 to 5, and presents them as a single account of one system rather than as a set of independent artefacts.

Each model in this report is accompanied by an explanation of what it shows, the requirement it serves, and the reasoning behind the choices made in it. Section 10 records the design decisions taken during domain analysis, and Section 11 records the inconsistencies still open at the time of this draft.

2. Problem statement

Students may experience delays in receiving their letters of completion after satisfying the requirements of their academic programme. The current process is manual: staff must retrieve a student's academic record, total the credits earned, compare those against programme requirements, and prepare a letter by hand. Each of these steps introduces delay, and the student has no visibility of progress while waiting.

The proposed system addresses this by providing a structured way to verify a student's academic requirements, check completed credits against programme requirements, identify outstanding requirements, support the approval of completion, generate a digital letter of completion, and deliver that letter electronically to the student.

3. Objectives

The system aims to:

1. Verify whether a student has fulfilled the requirements of their programme.
2. Calculate and validate the student's completed credits.
3. Identify any outstanding academic requirements.
4. Reduce delays associated with manual completion verification.
5. Support the generation of a digital completion letter.
6. Provide a reliable record of the completion verification process.

4. Scope

4.1 In scope

- A student submits a request for a letter of completion through the system.
- The system checks the student's records for completion status and credits earned.
- Staff review the request and approve or reject it.
- The system generates a letter of completion if approved, or records a refusal with its reason if not.
- The student receives and accesses the letter.

4.2 Out of scope

- Changing official university grades.
- Registering students for courses.
- Calculating tuition fees.
- Processing graduation ceremonies.
- Changing university policies.

4.3 Assumptions

- Authorised staff are responsible for final approval.
- Programme requirements are defined and available to the system.
- Students hold a university login or institutional email account.

4.4 Constraints

- The project must be completed within one semester.
- The prototype must not use confidential student records.
- The team has limited access to lecturer-approved technology.

4.5 Privacy considerations and risks

The system stores only what is needed to verify completion. Access is limited to the student concerned and authorised staff, and the data is used solely to verify completion and issue the letter.

Four risks are carried into design. The system could incorrectly classify a student as complete or incomplete; unauthorised parties could gain access to student records; the system could be unavailable when a student needs a letter; and records could be lost through technical failure. The first is addressed by the eligibility rules in Section 9, the second by the authentication requirement FR-01, and the third by Quality Scenario 4.

5. Stakeholders and actors

5.1 Stakeholders

- Students
- Academic departments
- Faculty and programme administrators
- Examinations and records officers
- University management
- IT and system administrators

5.2 Actors and their goals

Student — check their completion status; view completed credits; view outstanding credits; access their completion letter when eligible.

Academic staff — access and update student academic records; check a student's completion status; verify whether a student has satisfied programme requirements.

External verifier — confirm the authenticity of a letter presented to them (FR-14). This actor does not hold an account in the system and interacts only through a verification code.

6. Functional requirements

The system's functional requirements are recorded as FR-01 to FR-15 in `requirements.md`, each with an associated acceptance criterion. They fall into four groups:

Access and record retrieval (FR-01, FR-02). Authentication of students and staff with role-based access, and retrieval of a student's academic record.

Assessment (FR-03 to FR-08). Calculating credits earned, comparing them against the programme's requirement, checking compulsory modules and elective thresholds, itemising anything outstanding, and deriving an eligibility status without manual staff input.

Approval and issuance (FR-09 to FR-12). Staff review of an assessment, approval or rejection with a timestamp, generation of the letter on approval, and delivery to the student's institutional email address.

Verification and audit (FR-13 to FR-15). A unique verification code bound to each letter, external verification using that code, and an audit trail of approvals, letter generation and verification attempts.

FR-08 is worth noting as the hinge of the whole workflow: it requires the eligibility status to be produced automatically from FR-03 to FR-07, which is what allows the system to reduce delay rather than merely relocate it.

7. Quality scenarios

Four quality scenarios constrain the design beyond its functional behaviour.

Response time. A student requesting their completion status under normal usage should see it within three seconds.

Accuracy. A calculated credit total must exactly match the sum of credits assigned to the student's completed modules. This is the scenario that most directly addresses the risk of misclassifying a student.

Unauthorised access. An unauthenticated or unauthorised user attempting to reach a student's academic record must be shown no protected information.

Availability. The system should be available at least 99% of the time during the university's scheduled operating hours.

8. Use cases

Eleven use cases describe the system's behaviour from the actors' point of view. They are recorded in `use-cases.md`.

ID	Use case	Primary actor
UC-01	Authenticate user	Student
UC-02	Request completion letter	Student
UC-03	Verify completion status	System
UC-04	Confirm student clearance	Finance officer
UC-05	Approve or reject letter request	Registry officer
UC-06	Generate and issue digital letter	System
UC-07	Verify letter authenticity	External verifier
UC-08	Track request status	Student
UC-09	View credit summary	Student
UC-10	View outstanding requirements	Student
UC-11	Update academic record	Academic staff

UC-02 is the core workflow of the system and is the use case modelled in Section 9.3; it spans the student's request through verification (UC-03), approval (UC-05) and issuance (UC-06). UC-09 and UC-10 are read-only enquiries against the same assessment logic, and UC-08 lets a student follow a request already submitted. UC-11 is the staff-side counterpart that makes re-assessment possible: when a record is updated, a student previously found ineligible may become eligible. UC-07 serves the external verifier, who holds no account and interacts only through a verification code.

Each use case carries alternative flows for its important exceptional outcomes. UC-03 handles an academic record that cannot be retrieved; UC-05 handles rejection with a recorded reason; UC-07 handles an unknown verification code; UC-10 handles the case where nothing is outstanding. These alternatives are reflected in the behavioural models.

Open item. UC-04 introduces a financial clearance step performed by a Finance Officer, and step 4 of UC-02 depends on it. No requirement in FR-01 to FR-15 covers financial clearance, and Section 4.2 places the calculation of tuition fees out of scope. This use case therefore has no requirement behind it and is listed in Section 11 as unresolved.

8.1 Acceptance criteria

Four acceptance criteria state the conditions under which the system is judged to behave correctly: a student meeting all requirements is marked eligible; a student with outstanding requirements is marked incomplete and shown what is missing; an eligible student's letter is generated and made available; and a student whose record cannot be retrieved sees an error and receives no letter.

9. Analysis models

9.1 Domain model

The domain model (`models/domain-model.mmd`) identifies fourteen problem-domain concepts and the relationships between them. Its structure follows the workflow: a `Student` holds one or more `Enrolment` records, each governed by a `Programme` that defines a set of `CompletionRequirement` records. Results are held as `CourseResult` records under an enrolment. A `CompletionCheck` evaluates an enrolment against its programme's requirements. A `LetterRequest` triggers a check, is resolved by an `ApprovalDecision` made by a `RegistryOfficer`, and may produce a `CompletionLetter`. Each letter carries a `VerificationCode`, which answers `VerificationRequest` enquiries from external parties. `AuditEntry` records actions taken against a request.

Three modelling choices in this diagram are argued in Section 10: why `LetterRequest` is a class rather than a status field, why `CompletionCheck` attaches to `Enrolment` rather than to `Student`, and where composition is justified.

9.2 Responsibility allocation

Responsibilities are allocated in `docs/crc-cards.md` on the principle that a responsibility belongs to the class that already holds the information needed to carry it out. Credit calculation therefore sits with `Enrolment`, which holds the `CourseResult` records; the decision on whether a criterion is met sits with `CompletionRequirement`, which knows the criterion; and the eligibility outcome sits with `CompletionCheck`, which is the only class that sees both sides of the comparison.

9.3 Interaction model

The sequence diagram (`models/sequence-core-use-case.mmd`) models UC-02 from the student's submission through to a delivered letter.

The main flow proceeds: the student submits a request, which is recorded and audited; the request triggers a `CompletionCheck`; the check obtains completed results and the credit total from `Enrolment` and asks each `CompletionRequirement` whether it is satisfied; the outcome returns to the request. Where the outcome is eligible, a `RegistryOfficer` records an approving `ApprovalDecision`, a `CompletionLetter` is generated using a `LetterTemplate` and issued with a `VerificationCode`, and the letter is delivered to the student's institutional address.

The alternative flow is the case where the outcome is not eligible. Here no letter is produced, but an `ApprovalDecision` still records the refusal and its reason, and the student receives an itemised list of outstanding requirements rather than a bare failure. This branch is what FR-07 and FR-08 require, and it is also why the refusal must be recorded against a request that persists — the reasoning set out in Decision 1.

9.4 Lifecycle model

The state machine (`models/lifecycle-letter-request.mmd`) models the lifecycle of `LetterRequest`, the entity whose state drives the workflow.

A request moves from `Submitted` into `UnderVerification` when a check runs. From there it reaches `AwaitingApproval` when the check finds the student eligible, or `Refused` when it does not. An approved request becomes `LetterIssued` and then `Delivered`. Three exceptional outcomes are modelled: `VerificationFailed`, where the academic record cannot be retrieved and the check may be retried, taken from UC-03's alternative flow; `DeliveryFailed`, where delivery is not confirmed within the FR-12 window and is resent; and `Revoked`, where an issued letter is withdrawn and its verification code marked revoked.

The transition from `Refused` back to `Submitted` is the one that connects this model to UC-11: when academic staff update a record, a student previously refused may request again. That loop is only representable because request state is held on `LetterRequest` rather than on `Student`.

10. Design decisions

Four domain-modelling decisions are recorded in full in `decisions/D-002.md`, each stating the choice made, a realistic alternative and the consequence accepted.

`LetterRequest` is a class, not a status attribute on `Student`. A student may request a letter more than once, and FR-10 requires rejections to be recorded. Storing status on `Student` would allow only one request at a time, with each new request overwriting the last. The cost accepted is an additional class and an association to navigate.

`CompletionCheck` attaches to `Enrolment`, not to `Student`. Eligibility is judged against a programme's requirements, and a student may hold more than one enrolment. Attaching the check to `Student` would make the FR-08 outcome ambiguous for any student with two enrolments. The cost accepted is a longer navigation path for the most common query.

Composition is used only where lifetime ownership holds. `CompletionLetter` composes `VerificationCode`, since FR-13 binds a code permanently to one letter. `LetterRequest` only associates to `CompletionLetter`, because FR-14 requires an issued letter to remain verifiable independently of the request that produced it. The cost accepted is that referential integrity between request and letter must be enforced by rule.

`ApprovalDecision` is separate from `LetterRequest`. FR-09, FR-10 and FR-15 require an attributable, timestamped decision. Holding the outcome as attributes on the request would leave the deciding officer without a modelled relationship. The cost accepted is one more class and two more associations.

11. Consistency and open items

This draft is submitted with the following inconsistencies identified but not yet resolved. They are recorded here rather than concealed, since resolving them is the purpose of the Phase 1 review.

Vocabulary divergence between the CRC cards and the domain model. The CRC cards as currently committed use `Academic Record`, `Module`, `Module Result`, `Academic Staff` and `Programme Requirement`. The domain model uses `Enrolment`, `CourseResult`, `RegistryOfficer` and `CompletionRequirement`. Only `Student` and `CompletionLetter` appear in both. A revised set of cards using the domain model vocabulary has been prepared and is awaiting team agreement.

Two requirements files. `requirements.md` holds the canonical FR-01 to FR-15 table; `docs/requirements.md` holds an earlier draft of ten unnumbered statements. The earlier draft should be removed.

Empty traceability matrix. `docs/use cases/traceability-matrix.md` exists but has no content. The requirement-to-verification mapping required by Laboratory 4 is therefore outstanding.

Audit associations. The sequence diagram shows `ApprovalDecision` and `CompletionLetter` writing to `AuditEntry`, as FR-15 requires. The domain model currently associates `AuditEntry` only with `LetterRequest`. Either the model gains those associations or the audit responsibility is centralised on the request.

Use case for financial clearance. UC-04, and step 4 of UC-02, depend on a financial clearance check that no requirement covers and that the project scope excludes. Either FR-16 is added to cover it, or the use case and that step are removed.

Unmerged branch content. Stakeholder analysis material remains outside the default branch following the reverts of pull requests #11 and #12, and the Laboratory 2 problem documentation is still awaiting merge.

12. Evidence index

Artefact	Location
Functional requirements FR-01 to FR-15	`requirements.md`
Actors and actor goals	`docs/actors.md`
Use cases UC-01 to UC-11	`use-cases.md`
Acceptance criteria	`docs/use cases/acceptance-criteria.md`
Quality scenarios	`docs/use cases/quality-scenarios.md`
Domain model	`models/domain-model.mmd, .svg`
Business rules	`docs/business-rules.md`
Responsibility allocation	`docs/crc-cards.md`
Sequence diagram (UC-04)	`models/sequence-core-use-case.mmd`
Lifecycle state machine	`models/lifecycle-letter-request.mmd`